JDBC stored procedure

Almost stored procedures that you develop require parameters. The parameters make the stored procedure more flexible and useful.

In MySQL, a parameter has one of three modes: **IN**, **OUT**, or **INOUT**.

IN parameters

IN is the default mode. When you define an IN parameter in a stored procedure, the calling program has to pass an argument to the stored procedure. In addition, the value of an IN parameter is protected. It means that even the value of the IN parameter is changed inside the stored procedure, its original value is retained after the stored procedure ends. In other words, the stored procedure only works on the copy of the IN parameter.

OUT parameters

The value of an OUT parameter can be changed inside the stored procedure and its new value is passed back to the calling program. Notice that the stored procedure cannot access the initial value of the OUT parameter when it starts.
INOUT parameters

An INOUT parameter is a combination of IN and OUT parameters. It means that the calling program may pass the argument, and the stored procedure can modify the INOUT parameter, and pass the new value back to the calling program.

Defining a parameter

Here is the basic syntax of defining a parameter in stored procedures:

```
[IN | OUT | INOUT] parameter_name datatype[(length)]
```

In this syntax,

- First, specify the parameter mode, which can be IN , OUTor INOUT , depending on the purpose of the parameter in the stored procedure.
- Second, specify the name of the parameter. The parameter name must follow the naming rules of the column name in MySQL.
- Third, specify the data type and maximum length of the parameter.

Three types of parameters exist: IN, OUT, and INOUT. The PreparedStatement object only uses the IN parameter. The CallableStatement object can use all the three.

Here are the definitions of each –

Parameter	Description
IN	A parameter whose value is unknown when the SQL statement is created. You bind values to IN parameters with the setXXX() methods.
OUT	A parameter whose value is supplied by the SQL statement it returns. You retrieve values from the OUT parameters with the getXXX() methods.
INOUT	A parameter that provides both input and output values. You bind variables with the setXXX() methods and retrieve values with the getXXX() methods.

How to get the instance of CallableStatement?

The prepareCall() method of Connection interface returns the instance of CallableStatement. Syntax is given below:

```
1. public CallableStatement prepareCall("{ call procedurename(?,?...?)}");
```

The example to get the instance of CallableStatement is given below:

```
1. CallableStatement stmt=con.prepareCall("{call myprocedure(?,?)}");
```

```
CREATE TABLE `employees` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `last_name` varchar(64) DEFAULT NULL,
  `first_name` varchar(64) DEFAULT NULL,
  `email` varchar(64) DEFAULT NULL,
  `department` varchar(64) DEFAULT NULL,
  `salary` DECIMAL(10,2) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=latin1;

INSERT INTO `employees` (`id`,`last_name`,`first_name`,`email`, `department`, `salary`) VALUES (1,'Doe','John','john.doe@foo.com', 'HR', 55000.00);
INSERT INTO `employees` (`id`,`last_name`,`first_name`,`email`, `department`, `salary`) VALUES (2,'Public','Mary','mary.public@foo.com', 'Engineering', 75000.00);
INSERT INTO `employees` (`id`,`last_name`,`first_name`,`email`, `department`, `salary`) VALUES (3,'Queue','Susan','susan.queue@foo.com', 'Legal', 130000.00);

INSERT INTO `employees` (`id`,`last_name`,`first_name`,`email`, `department`, `salary`) VALUES (4,'Williams','David','david.williams@foo.com', 'HR', 120000.00);
INSERT INTO `employees` (`id`,`last_name`,`first_name`,`email`, `department`, `salary`) VALUES (5,'Johnson','Lisa','lisa.johnson@foo.com', 'Engineering', 50000.00);
INSERT INTO `employees` (`id`,`last_name`,`first_name`,`email`, `department`, `salary`) VALUES (6,'Smith','Paul','paul.smith@foo.com', 'Legal', 100000.00);

INSERT INTO `employees` (`id`,`last_name`,`first_name`,`email`, `department`, `salary`) VALUES (7,'Adams','Carl','carl.adams@foo.com', 'HR', 50000.00);
INSERT INTO `employees` (`id`,`last_name`,`first_name`,`email`, `department`, `salary`) VALUES (8,'Brown','Bill','bill.brown@foo.com', 'Engineering', 50000.00);
INSERT INTO `employees` (`id`,`last_name`,`first_name`,`email`, `department`, `salary`) VALUES (9,'Thomas','Susan','susan.thomas@foo.com', 'Legal', 80000.00);

INSERT INTO `employees` (`id`,`last_name`,`first_name`,`email`, `department`, `salary`) VALUES (10,'Davis','John','john.davis@foo.com', 'HR', 45000.00);
INSERT INTO `employees` (`id`,`last_name`,`first_name`,`email`, `department`, `salary`) VALUES (11,'Fowler','Mary','mary.fowler@foo.com', 'Engineering', 65000.00);
INSERT INTO `employees` (`id`,`last_name`,`first_name`,`email`, `department`, `salary`) VALUES (12,'Waters','David','david.waters@foo.com', 'Legal', 90000.00);
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id	last_name	first_name	email	department	salary
1	Doe	John	john.doe@foo.com	HR	55000.00
2	Public	Mary	mary.public@foo.com	Engineering	115000.00
3	Queue	Susan	susan.queue@foo.com	Legal	130000.00
4	Williams	David	david.williams@foo.com	HR	120000.00
5	Johnson	Lisa	lisa.johnson@foo.com	Engineering	90000.00
6	Smith	Paul	paul.smith@foo.com	Legal	100000.00
7	Adams	Carl	carl.adams@foo.com	HR	50000.00
8	Brown	Bill	bill.brown@foo.com	Engineering	90000.00
9	Thomas	Susan	susan.thomas@foo.com	Legal	80000.00
10	Davis	John	john.davis@foo.com	HR	45000.00
11	Fowler	Mary	mary.fowler@foo.com	Engineering	105000.00
12	Waters	David	david.waters@foo.com	Legal	90000.00

--
-- DEFINE STORED PROCEDURES
--

➢ Creating CallableStatement Object

```
USE `college`;
DROP procedure IF EXISTS `increase_salaries_for_department`;

DELIMITER $$
USE `college`$$
CREATE PROCEDURE `increase_salaries_for_department`(IN the_department VARCHAR(64), IN increase_amount DECIMAL(10,2))
BEGIN
	UPDATE EMPLOYEES SET salary= salary + increase_amount where department=the_department;
END$$
DELIMITER ;
```

Program 1 (IN Mode)

```
import java.sql.*;
public class increasesalaries {

   public static void main(String[] args) throws Exception {

		Connection myConn = null;
		CallableStatement myStmt = null;

		try {
			// Get a connection to database
			myConn = DriverManager.getConnection("jdbc:mysql://localhost:3306/college",
"root", "root");

			String theDepartment = "Engineering";
			int theIncreaseAmount = 10000;

			// Show salaries BEFORE
			System.out.println("Salaries BEFORE\n");
			showSalaries(myConn, theDepartment);

			// Prepare the stored procedure call
			myStmt = myConn.prepareCall("{call increase_salaries_for_department(?, ?)}");

			// Set the parameters
			myStmt.setString(1, theDepartment);
			myStmt.setDouble(2, theIncreaseAmount);

			// Call stored procedure
```

```
		System.out.println("\n\nCalling stored procedure.
increase_salaries_for_department('" + theDepartment + "', " + theIncreaseAmount + ")");
		myStmt.execute();
		System.out.println("Finished calling stored procedure");

		// Show salaries AFTER
		System.out.println("\n\nSalaries AFTER\n");
		showSalaries(myConn, theDepartment);

	} catch (Exception exc) {
		exc.printStackTrace();
	} finally {
		close(myConn, myStmt, null);
	}
}

private static void showSalaries(Connection myConn, String theDepartment) throws SQLException {
	PreparedStatement myStmt = null;
	ResultSet rs = null;

	try {
		// Prepare statement
		myStmt = myConn.prepareStatement("select * from employees where
department=?");

		myStmt.setString(1, theDepartment);

		// Execute SQL query
		rs = myStmt.executeQuery();

		// Process result set
		while (rs.next()) {
			String lastName = rs.getString("last_name");
			String firstName = rs.getString("first_name");
			double salary = rs.getDouble("salary");
			String department = rs.getString("department");

			System.out.printf("%s, %s, %s, %.2f\n", lastName, firstName, department,
salary);
		}
	} catch (Exception exc) {
		exc.printStackTrace();
	} finally {
		close(myStmt, rs);
	}

}

private static void close(Connection myConn, Statement myStmt, ResultSet rs) throws SQLException {
	if (rs != null) {
		rs.close();
	}

	if (myStmt != null) {
		myStmt.close();
	}
```

```
            if (myConn != null) {
                        myConn.close();
            }
    }

    private static void close(Statement myStmt, ResultSet rs)
                        throws SQLException {

            close(null, myStmt, rs);
    }
}
```

OUTPUT:

```
Output - CallSt (run)
run:
Salaries BEFORE

Public, Mary, Engineering, 105000.00
Johnson, Lisa, Engineering, 80000.00
Brown, Bill, Engineering, 80000.00
Fowler, Mary, Engineering, 95000.00


Calling stored procedure.  increase_salaries_for_department('Engineering', 10000)
Finished calling stored procedure


Salaries AFTER

Public, Mary, Engineering, 115000.00
Johnson, Lisa, Engineering, 90000.00
Brown, Bill, Engineering, 90000.00
Fowler, Mary, Engineering, 105000.00
BUILD SUCCESSFUL (total time: 1 second)
```

--
-- DEFINE STORED PROCEDURES
--

```
USE `college`;
DROP procedure IF EXISTS `greet_the_department`;

DELIMITER $$
USE `college`$$
CREATE PROCEDURE `greet_the_department`(INOUT department VARCHAR(64))
BEGIN

    SET department = concat('Hello to the awesome ', department, ' team!');

END$$

DELIMITER ;
```

Program 2 (INOUT Mode)

```
import java.sql.*;

public class GreetTheDepartment {

   public static void main(String[] args) throws Exception {

            Connection myConn = null;
            CallableStatement myStmt = null;

            try {
                    // Get a connection to database
                    myConn = DriverManager.getConnection("jdbc:mysql://localhost:3306/college",
"root", "root");

                    String theDepartment = "Engineering";

                    // Prepare the stored procedure call
                    myStmt = myConn.prepareCall("{call greet_the_department(?)}");

                    // Set the parameters
                    myStmt.registerOutParameter(1, Types.VARCHAR);
                    myStmt.setString(1, theDepartment);

                    // Call stored procedure
                    System.out.println("Calling stored procedure.greet_the_department('" +
theDepartment + "')");
                    myStmt.execute();
                    System.out.println("Finished calling stored procedure");
```

```
                // Get the value of the INOUT parameter
                String theResult = myStmt.getString(1);

                System.out.println("\nThe result = " + theResult);

        } catch (Exception exc) {
                exc.printStackTrace();
        } finally {
                close(myConn, myStmt);
        }
    }

    private static void close(Connection myConn, Statement myStmt) throws SQLException {
        if (myStmt != null) {
                myStmt.close();
        }

        if (myConn != null) {
                myConn.close();
        }
    }
}
```

OUTPUT:

```
run:
Calling stored procedure.  greet_the_department('Engineering')
Finished calling stored procedure

The result = Hello to the awesome Engineering team!
BUILD SUCCESSFUL (total time: 1 second)
```

```
--
-- DEFINE STORED PROCEDURES
--
```

```
USE `college`;
DROP procedure IF EXISTS `get_count_for_department`;

DELIMITER $$
USE `college`$$
CREATE PROCEDURE `get_count_for_department`(IN the_department VARCHAR(64), OUT the_count
INT)
BEGIN
        SELECT COUNT(*) INTO the_count FROM employees where department=the_department;
END$$
DELIMITER ;
```

Program-3 (OUT Mode)

```
import java.sql.CallableStatement;

import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.SQLException;

import java.sql.Statement;

import java.sql.Types;

public class GetCountForDepartment {

        public static void main(String[] args) throws Exception {

                Connection myConn = null;

                CallableStatement myStmt = null;

                try {

                        // Get a connection to database

                        myConn = DriverManager.getConnection("jdbc:mysql://localhost:3306/college",
"root", "root");

                        String theDepartment = "Engineering";


                        // Prepare the stored procedure call

                        myStmt = myConn.prepareCall("{call get_count_for_department(?, ?)}");
```

```
                // Set the parameters
                myStmt.setString(1, theDepartment);
                myStmt.registerOutParameter(2, Types.INTEGER);

                // Call stored procedure
                System.out.println("Calling stored procedure. get_count_for_department('" + theDepartment + "', ?)");
                myStmt.execute();
                System.out.println("Finished calling stored procedure");

                // Get the value of the OUT parameter
                int theCount = myStmt.getInt(2);

                System.out.println("\nThe count = " + theCount);

            } catch (Exception exc) {
                exc.printStackTrace();
            } finally {
                close(myConn, myStmt);
            }
        }
        private static void close(Connection myConn, Statement myStmt) throws SQLException {
            if (myStmt != null) {
                myStmt.close();
            }
            if (myConn != null) {
                myConn.close();
            }
        }
}
```

OUTPUT:

Output - CallSt (run)

```
run:
Calling stored procedure.  get_count_for_department('Engineering', ?)
Finished calling stored procedure

The count = 4
BUILD SUCCESSFUL (total time: 1 second)
```